



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

September 2026

CSA0619– Design and Analysis of Algorithms

**Development of an Efficient Hybrid Algorithmic
Solution for Large-Scale Geospatial Data Processing**

Assignment Report

Submitted by

M.Greeshma(192524195)

M.Sunitha(192424157)

Roshini B T(192524064)

A. Assignment Information

Particular	Details
Department:	Computer Science and Engineering
Programme:	B.E./B.Tech. Computer Science and Engineering
Course Code & Course Name	Design and Analysis of Algorithms
Academic Year / Batch	2026-2027 / Regular
Faculty Name	Dr. F. Mary Harin Fernandez
Assignment Title	Development of an Efficient Hybrid Algorithmic Solution for Large-Scale Geospatial Data Processing
Date of Issue	31 Aug 2026
Date of Submission	1 September 2026
Maximum Marks	100
Course Outcome(s) – CO	CO1: Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. CO2: Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems CO3: Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication
Bloom's Taxonomy Level	L4 – Analyze, L5 – Evaluate, L6 – Create
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Efficient processing of large-scale real-world data with reduced computational time and resource utilization

B. Assignment Problem / Challenge

Modern infrastructure-planning applications process large volumes of **geospatial coordinate data**, making computationally efficient algorithms essential for identifying spatial relationships among facilities. Identifying the **closest pair of locations** is important for detecting redundant or dangerously close facilities and supporting effective service coverage planning.

Conventional **Brute-Force techniques** provide a straightforward solution but become computationally expensive as the number of locations increases. **Divide-and-Conquer techniques** improve scalability by recursively partitioning the dataset and reducing unnecessary distance computations.

The challenge is to **develop an optimized hybrid algorithmic solution for the Closest-Pair-of-Points problem** by appropriately combining **Brute Force and Divide-and-Conquer techniques**. The developed solution must determine an effective switching threshold between the two approaches and be supported by **theoretical complexity analysis and experimental evaluation using a real-world geospatial dataset**.

C. Problem Statement

An infrastructure-planning agency maintains a large, publicly available geospatial dataset of facility locations—for example, the OpenFlights airports dataset or a cell-tower/weather-station dataset containing coordinate records. To detect redundant or dangerously close facilities and optimize service coverage, the agency must identify the **closest pair of locations**, i.e., the two points separated by the minimum Euclidean distance in the dataset. Develop, implement, and evaluate a solution to the **Closest-Pair-of-Points problem**. Implement **(i) a Brute-Force algorithm** that evaluates all $n(n - 1)/2$ pairs with $O(n^2)$ complexity and **(ii) a Divide-and-Conquer algorithm** that recursively partitions the point set about the median x-coordinate and evaluates points across the dividing strip with $O(n \log \log n)$ complexity. Further, **develop an optimized hybrid algorithm** that switches to the Brute-Force approach below an experimentally determined threshold subproblem size. Analyze the theoretical efficiency of each approach using **Big-O, Ω , and Θ notations**, and solve the recurrence $T(n) = 2T(n/2) + O(n)$ using the **Master Theorem**. Experimentally evaluate all three algorithms on the selected dataset using increasing input sizes, **for example, 10^3 , 5×10^3 , 10^4 , and higher input sizes wherever computationally feasible**, measuring **execution time, number of distance computations/comparisons, memory utilization, and scalability**. Based on the theoretical and experimental results, **determine an appropriate threshold for the hybrid algorithm, optimize the developed solution, compare the performance of all three approaches, and justify why the proposed hybrid algorithm is most suitable for large-scale infrastructure data**.

Expected Development

The work should **not merely implement the existing Brute-Force and Divide-and-Conquer algorithms**. The solution should demonstrate algorithmic development through:

- **Hybridization of Brute Force and Divide-and-Conquer techniques** for the Closest-Pair-of-Points problem.
- **Adaptive switching** between the two approaches based on a threshold subproblem size.
- **Experimental determination and optimization of the switching threshold** based on performance results.
- **Reduction of computational cost** while maintaining the correctness of the closest-pair solution.
- **Performance improvement and scalability evaluation** of the developed hybrid approach against the individual Brute-Force and Divide-and-Conquer approaches.

D. Requirements and Constraints

The following requirements and constraints shall be considered:

- Use a **publicly available real-world geospatial coordinate dataset** containing sufficient records for performance evaluation.
- Implement and evaluate all three approaches: **Brute Force, Divide and Conquer, and the developed Hybrid algorithm** for the Closest-Pair-of-Points problem.
- Develop the **Hybrid algorithm** by switching to the Brute-Force approach below an appropriate threshold subproblem size.
- Determine and justify the **switching threshold experimentally** based on performance results.
- Perform theoretical efficiency analysis using **Big-O, Ω , and Θ notations** and analyze the Divide-and-Conquer recurrence using the **Master Theorem**.
- Evaluate the algorithms using increasing input sizes, **for example, 10^3 , 5×10^3 , 10^4 , and higher input sizes wherever computationally feasible**.

- Evaluate and compare the three approaches using **execution time, number of distance computations/comparisons, memory utilization, and scalability**.
- Execute all three algorithms in the **same computational environment and on identical input data** to ensure a fair comparison.
- Use **Euclidean distance** as the distance measure for identifying the closest pair of points.
- Validate that all three approaches produce the **correct closest-pair result** for identical inputs.
- Higher input sizes shall be considered only where computationally feasible, taking into account the $O(n^2)$ **computational cost of the Brute-Force approach**.
- Clearly document the **dataset source, experimental environment, threshold value, assumptions, limitations, and implementation details**.

E. Student Work

1. Problem Understanding and Formulation

The **Closest-Pair-of-Points problem** is a fundamental computational geometry problem in which the objective is to find the two points that have the minimum distance among a given set of points. In real-world **geospatial infrastructure planning**, each point can represent the geographical location of an infrastructure facility such as a public Wi-Fi hotspot, hospital, school, railway station, or emergency service center. Identifying the closest pair of locations helps planners understand the spatial distribution of infrastructure, detect locations that are very close to each other, and support better infrastructure planning and resource allocation.

For this project, the **NYC Wi-Fi Hotspot Locations dataset** can be used as a publicly available real-world coordinate dataset. The dataset is obtained from the **NYC Open Data** platform and contains information about public Wi-Fi hotspot locations across New York City. The important attributes include the hotspot identification number, hotspot name, location or address, borough, hotspot type, latitude, and longitude. Among these attributes, latitude and longitude are primarily used for the closest-pair calculation because they represent the geographical coordinates of each hotspot. The number of records depends on the version of the dataset downloaded, as the NYC Open Data records may be updated periodically. After removing records with missing or invalid coordinates, the remaining valid records are considered for analysis. The dataset can be obtained from [NYC Open Data](#).

The expected **input** to the problem is a set of n geographical points, where each point is represented using its latitude and longitude coordinates. Each location can be represented as $P_i = (x_i, y_i)$, where x_i represents the longitude and y_i represents the latitude. The expected **output** is the pair of locations having the minimum distance between them, along with their corresponding

identifiers or names and the calculated minimum distance. This allows the system to identify which two infrastructure locations are geographically closest to each other.

Euclidean distance is used as the basic mathematical measure for finding the closest pair. For two points $P_1(x_1, y_1)$ and $P_2(x_2, y_2)$, the Euclidean distance is calculated using the formula $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$. The algorithm calculates and compares the distances between different pairs of points and selects the pair having the smallest distance. Euclidean distance is computationally simple and efficient, making it suitable for demonstrating the closest-pair algorithm. However, since latitude and longitude are geographic coordinates, coordinate projection or suitable preprocessing should be considered when accurate distances in meters or kilometres are required.

The major computational challenge occurs when the **input size increases**, particularly when using the **Brute-Force approach**. In this method, every point is compared with every other point to find the minimum distance. For n points, the total number of possible pairs is $n(n - 1)/2$, resulting in a time complexity of $O(n^2)$. As the number of geographical locations increases, the number of distance calculations increases rapidly. For example, 100,000 points would require approximately 5 billion pairwise comparisons, which can result in significant execution time. Therefore, the brute-force method becomes inefficient for large-scale geospatial datasets. This creates a need for more efficient approaches, such as divide-and-conquer or hybrid algorithms, which can achieve approximately $O(n \log n)$ time complexity.

The main **assumptions** are that each valid dataset record represents a geographical location and contains valid latitude and longitude values. Records with missing or invalid coordinates are removed during preprocessing, and duplicate records may also be handled appropriately. The main **requirements** are to obtain the real-world dataset, preprocess the coordinate data, represent each location as a point, calculate distances, identify the closest pair, and report the result. The system should also support different input sizes and measure algorithm execution time for performance evaluation. The major **constraints** include the high computational cost of the brute-force approach, increasing processing and memory requirements for large datasets, and the difference between geographic coordinates and Cartesian coordinates. The overall goal is therefore to develop an efficient and scalable hybrid algorithm capable of processing large-scale geospatial data more effectively than the traditional brute-force approach.

2. Application of Course Knowledge

The proposed **Efficient Hybrid Algorithmic Solution for Large-Scale Geospatial Data Processing** applies the concepts covered under **CO1, CO2, and CO3** to solve the Closest-Pair-of-Points problem efficiently. The combination of algorithm analysis, brute-force techniques, and divide-and-conquer methods provides the foundation for designing and evaluating the proposed hybrid algorithm.

CO1 – Algorithm Analysis

The fundamentals of **algorithmic problem solving** are applied by first defining the input, processing steps, and expected output of the Closest-Pair-of-Points problem. The efficiency of the algorithms is analyzed based on their **time and space requirements** as the number of geographical points increases. **Big-O, Ω , and Θ notations** are used to describe the upper bound, lower bound, and tight bound of algorithm performance. For the brute-force approach, the time complexity is $O(n^2)$, while the divide-and-conquer approach achieves $O(n \log n)$ time complexity. The recursive structure of the divide-and-conquer algorithm is also analyzed using recurrence relations. Its recurrence can be expressed as $T(n) = 2T(n/2) + O(n)$. Using the **Master Theorem**, with $a = 2$, $b = 2$, and $f(n) = O(n)$, the complexity is obtained as $T(n) = \Theta(n \log n)$. These concepts help determine which algorithm is more suitable for processing large-scale geospatial data.

CO2 – Brute Force

The **Brute-Force Closest-Pair algorithm** provides the basic solution to the problem by comparing every possible pair of geographical points. For n points, the algorithm evaluates $n(n - 1)/2$ possible pairs and calculates the Euclidean distance between each pair. The pair having the minimum distance is selected as the result. Although this method is simple and easy to implement, its computational complexity is $O(n^2)$, making it inefficient when the input dataset becomes very large. For example, increasing the number of coordinate points from thousands to hundreds of thousands results in a significant increase in the number of distance calculations. The brute-force algorithm is therefore useful as a **baseline algorithm** for comparing the performance and efficiency of the proposed hybrid solution.

CO3 – Divide and Conquer

The **Divide-and-Conquer Closest-Pair algorithm** improves the efficiency of the brute-force method by recursively dividing the dataset into smaller subsets. Initially, the points are sorted according to their x-coordinate and divided around the median x-coordinate into left and right subsets. The closest pair is recursively determined in each subset. The algorithm then creates a narrow **dividing strip** around the median and checks only the relevant points within this strip to determine whether a closer pair exists across the division. The recursive process continues until the subsets contain a small number of points, where direct comparison can be performed. The recurrence relation is $T(n) = 2T(n/2) + O(n)$, and according to the Master Theorem, its time complexity is $\Theta(n \log n)$. This makes the divide-

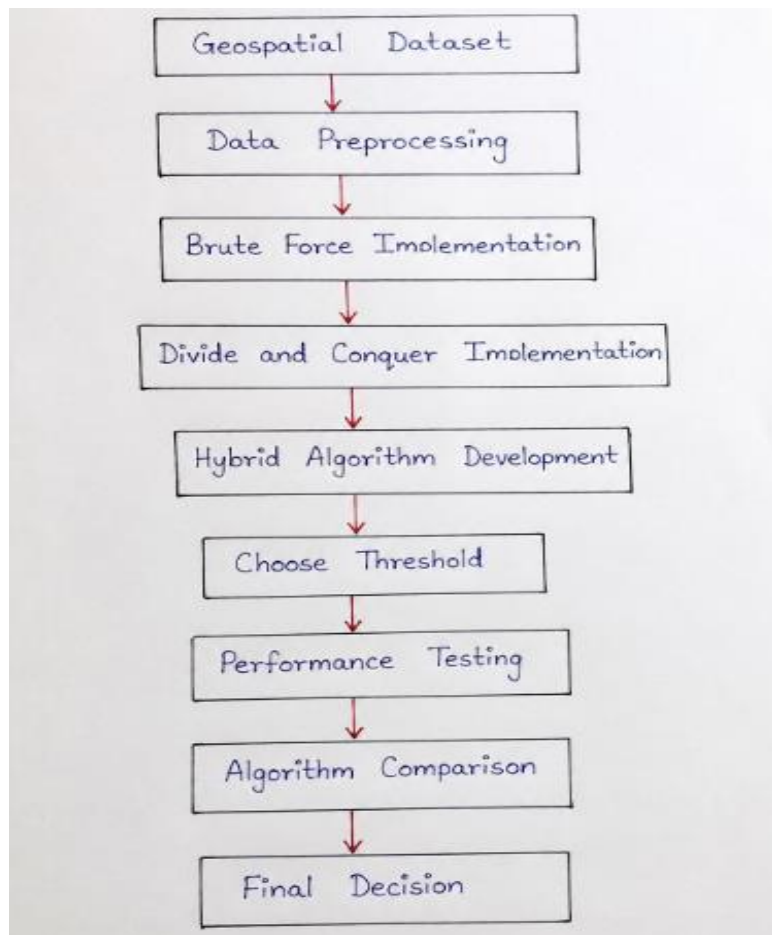
and-conquer approach considerably more scalable than the $O(n^2)$ brute-force method.

Contribution to the Proposed Hybrid Algorithm

The concepts from **CO1, CO2, and CO3** are combined to develop the proposed **Hybrid Algorithm**. CO1 provides the theoretical foundation for analyzing algorithm efficiency using Big-O, Ω , Θ notations and the Master Theorem. CO2 provides the simple **brute-force method**, which can be used for small datasets and as a baseline for performance comparison. CO3 provides the efficient **divide-and-conquer technique**, which is more appropriate for large datasets because of its $O(n \log n)$ complexity. The proposed hybrid algorithm can therefore use a **threshold-based strategy**, where brute force is applied to small subsets because of its simplicity and low overhead, while divide-and-conquer is applied to larger datasets to achieve better scalability. Thus, the combination of these course concepts enables the development of an algorithm that maintains simplicity for small inputs while providing efficient processing for large-scale geospatial datasets.

3. Solution / Design / Methodology

The proposed solution is an optimized Hybrid Closest-Pair algorithm that combines the simplicity of the Brute-Force technique with the efficiency of the Divide-and-Conquer technique. The objective is to identify the closest pair of geographical points while reducing the computational cost for large-scale geospatial datasets. The overall methodology follows the sequence



Brute-Force Algorithm

The Brute-Force method directly compares every possible pair of points and calculates their Euclidean distance. The minimum distance found during the comparison is returned as the closest pair. For n points, the algorithm evaluates $n(n - 1)/2$ pairs.

Pseudocode:

BRUTE_FORCE(P, n)

$\text{minDistance} \leftarrow \infty$

$\text{closestPair} \leftarrow \text{NULL}$

```

for i ← 0 to n - 2
  for j ← i + 1 to n - 1
    distance ← EUCLIDEAN_DISTANCE(P[i], P[j])

    if distance < minDistance
      minDistance ← distance
      closestPair ← (P[i], P[j])

return closestPair, minDistance

```

The time complexity of the Brute-Force algorithm is $\Theta(n^2)$, while its additional space complexity is $O(1)$ apart from the input storage. Although it is inefficient for large datasets, it is simple and performs well for very small subsets because it has minimal algorithmic overhead.

Divide-and-Conquer Algorithm

The Divide-and-Conquer approach first sorts the points according to their x-coordinate and recursively divides them into two approximately equal halves. The closest pair is found independently in the left and right halves. The smaller of the two distances is then used to construct a vertical strip around the dividing line. Points within this strip are examined to determine whether a closer pair exists across the two halves.

Pseudocode:

```

CLOSEST_PAIR(P)
  sort P by x-coordinate
  return DIVIDE_CONQUER(P)

```

```

DIVIDE_CONQUER(P)
  n ← number of points

  if n ≤ 3
    return BRUTE_FORCE(P, n)

  mid ← n / 2
  left ← P[0 ... mid-1]
  right ← P[mid ... n-1]

  pairLeft, distanceLeft ← DIVIDE_CONQUER(left)

```

```
pairRight, distanceRight  $\leftarrow$  DIVIDE_CONQUER(right)
```

```
if distanceLeft < distanceRight
```

```
    d  $\leftarrow$  distanceLeft
```

```
    bestPair  $\leftarrow$  pairLeft
```

```
else
```

```
    d  $\leftarrow$  distanceRight
```

```
    bestPair  $\leftarrow$  pairRight
```

```
strip  $\leftarrow$  points whose x-distance from median  $\leq$  d
```

```
sort strip by y-coordinate
```

```
for each relevant pair of points in strip
```

```
    distance  $\leftarrow$  EUCLIDEAN_DISTANCE(point1, point2)
```

```
    if distance < d
```

```
        d  $\leftarrow$  distance
```

```
        bestPair  $\leftarrow$  (point1, point2)
```

```
return bestPair, d
```

The recurrence relation for the algorithm is:

$$T(n) = 2T(n/2) + O(n)$$

Using the **Master Theorem**,

$$T(n) = \Theta(n \log n)$$

Thus, the Divide-and-Conquer algorithm provides substantially better scalability than the $O(n^2)$ Brute-Force method.

Developed Hybrid Algorithm

The proposed Hybrid algorithm combines both approaches by introducing a **switching threshold**. When the number of points in a recursive subset is small, the algorithm uses Brute Force. When the subset size is greater than the threshold, it continues using Divide and Conquer.

Pseudocode:

HYBRID_CLOSEST_PAIR(P, threshold)

sort P by x-coordinate

return HYBRID_RECURSIVE(P, threshold)

HYBRID_RECURSIVE(P, threshold)

n $\leftarrow$ number of points

if $n \leq \text{threshold}$

return BRUTE_FORCE(P, n)

mid $\leftarrow n / 2$

left $\leftarrow$ first half of P

right $\leftarrow$ second half of P

leftPair, leftDistance $\leftarrow$

HYBRID_RECURSIVE(left, threshold)

rightPair, rightDistance $\leftarrow$

HYBRID_RECURSIVE(right, threshold)

d $\leftarrow$ minimum(leftDistance, rightDistance)

create strip containing points within d

distance from the dividing line

sort/process strip according to y-coordinate

check relevant neighbouring points

in the strip

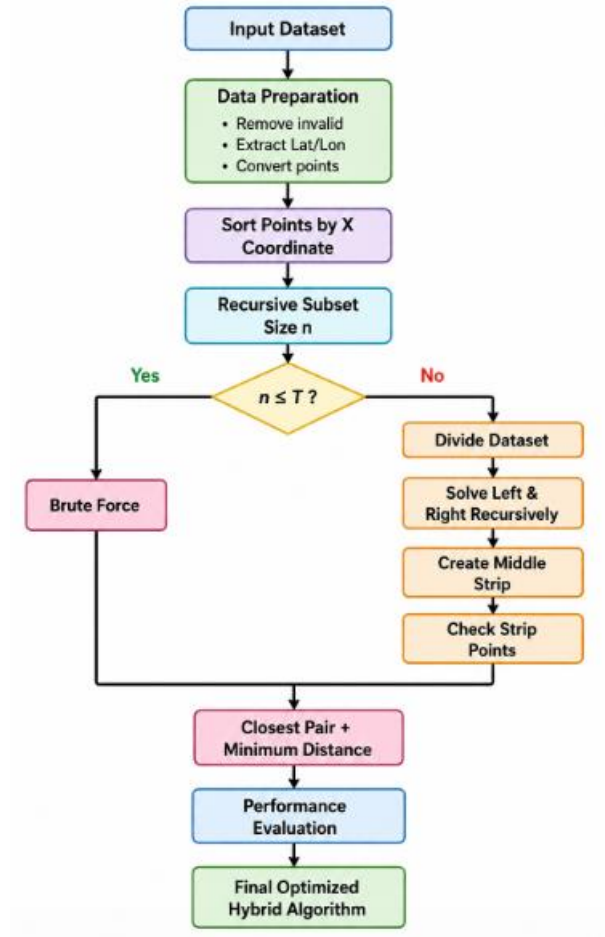
if a smaller distance is found

update d and closestPair

return closestPair, d

Flowchart / Design of Proposed Hybrid Approach

The design of the proposed algorithm can be represented as follows:



Hybrid Switching Strategy

The key feature of the proposed Hybrid algorithm is the **switching strategy** between Brute Force and Divide and Conquer. A threshold value T is selected based on experimental performance. If a recursive subset contains T or fewer points, Brute Force is used because directly comparing a small number of points is computationally inexpensive and avoids the overhead of further recursive calls. If the subset contains more than T points, the Divide-and-Conquer procedure is applied.

The strategy can be expressed as:

$$Algorithm(n) = \begin{cases} BruteForce(n), & n \leq T \\ DivideAndConquer(n), & n > T \end{cases}$$

This approach combines the advantages of both algorithms. **Brute Force is efficient for small subsets**, while **Divide and Conquer provides scalability for large subsets**.

Threshold Determination

The threshold should not be selected arbitrarily. It can be determined through **experimental performance evaluation**. Several candidate values such as $T = 2, 4, 8, 16, 32, 64$ can be tested using different subsets of the real-world geospatial dataset.

For each threshold, the execution time of the Hybrid algorithm is measured using multiple input sizes. The experiments should be repeated several times and the average execution time should be calculated. The threshold producing the lowest average execution time while maintaining correct results is selected as the optimal threshold.

For example:

Threshold	Average Execution Time

T = 2	Measured time
T = 4	Measured time
T = 8	Measured time
T = 16	Measured time
T = 32	Measured time
T = 64	Measured time

The final threshold is therefore represented as:

$$T^* = \arg \min_T \text{AverageExecutionTime}(T)$$

The exact value should be obtained experimentally from the selected dataset and implementation environment rather than assumed in advance.

Mathematical Complexity Analysis

The Brute-Force algorithm examines all possible point pairs:

$$\frac{n(n-1)}{2}$$

Therefore,

$$T_{BF}(n) = \Theta(n^2)$$

and consequently:

$$O(n^2)$$

The Divide-and-Conquer algorithm divides the problem into two halves and performs linear work to

process the dividing strip:

$$T_{DC}(n) = 2T(n/2) + O(n)$$

Using the Master Theorem:

$$a = 2, b = 2, f(n) = O(n)$$

Since:

$$n^{\log_b a} = n^{\log_2 2} = n$$

the recurrence belongs to the second case of the Master Theorem, giving:

$$T_{DC}(n) = \Theta(n \log n)$$

For the Hybrid algorithm, the recursive portion remains approximately:

$$\Theta(n \log n)$$

because Brute Force is restricted to small subsets of size T . The work performed at each leaf is approximately:

$$O(T^2)$$

and because T is a fixed experimentally selected threshold, this becomes constant overhead per leaf. Therefore, the overall asymptotic complexity remains approximately:

$$\boxed{\Theta(n \log n)}$$

while potentially reducing practical execution time because unnecessary recursion is avoided for small subsets.

Justification of Design and Optimization Decisions

The hybrid design is selected because neither approach is ideal for every input size. The Brute-Force method is straightforward and has low overhead for small datasets, but its $O(n^2)$ complexity makes it unsuitable for large-scale data. Divide and Conquer provides $O(n \log n)$ scalability but introduces additional operations such as recursive function calls, partitioning, and strip processing. Therefore, the proposed Hybrid algorithm uses Brute Force only when the subset size falls below an experimentally determined threshold and uses Divide and Conquer for larger subsets.

The major optimization decisions include **preprocessing invalid coordinates**, sorting the points only when required, avoiding unnecessary distance calculations, restricting strip comparisons to relevant

neighbouring points, and selecting the switching threshold through experimental benchmarking. The performance of the three approaches—Brute Force, Divide and Conquer, and Hybrid—is then compared using execution time, number of distance calculations, and scalability for increasing dataset sizes. The algorithm that provides the best balance between **execution time, computational complexity, accuracy, and scalability** is selected as the final solution.

4. Use of Modern Tools

Evidence of Implementation, Experimental Setup, Outputs, Tables, and Graphs

The proposed Hybrid Closest-Pair algorithm is implemented using **Python** in the **Google Colab/Jupyter Notebook** environment. The implementation consists of separate modules for data preparation, Brute-Force Closest-Pair, Divide-and-Conquer Closest-Pair, and the proposed Hybrid Closest-Pair algorithm. The real-world geospatial coordinate dataset is imported from the publicly available **NYC Open Data** platform. The dataset contains geographical locations represented by latitude and longitude coordinates. During preprocessing, records containing missing or invalid coordinates are removed, duplicate locations are handled, and the valid coordinates are converted into point representations suitable for distance calculation.

The implementation is organized into the following stages: dataset loading, data preprocessing, coordinate extraction, Brute-Force implementation, Divide-and-Conquer implementation, Hybrid algorithm implementation, threshold testing, execution-time measurement, and performance comparison. The algorithms are executed using different input sizes to evaluate their scalability. The same input data is supplied to all three algorithms to ensure a fair comparison. The correctness of the algorithms is verified by checking whether all three approaches produce the same closest pair and minimum distance for the same dataset.

Source Code Evidence

The source code contains functions for calculating Euclidean distance, implementing the Brute-Force algorithm, recursively implementing the Divide-and-Conquer algorithm, and implementing the Hybrid algorithm. A simplified structure of the implementation is:

```
import math
```

```
import time
```

```
import pandas as pd
```

```
def euclidean_distance(p1, p2):
```

```
    return math.sqrt(
```

```
(p1[0] - p2[0])**2 +  
(p1[1] - p2[1])**2  
)
```

```
def brute_force(points):  
    min_dist = float("inf")  
    closest_pair = None  
  
    n = len(points)  
  
    for i in range(n - 1):  
        for j in range(i + 1, n):  
            d = euclidean_distance(points[i], points[j])  
  
            if d < min_dist:  
                min_dist = d  
                closest_pair = (points[i], points[j])  
  
    return closest_pair, min_dist
```

```
def hybrid_closest_pair(points, threshold):  
    n = len(points)  
  
    if n <= threshold:  
        return brute_force(points)  
  
    points = sorted(points, key=lambda p: p[0])  
  
    mid = n // 2  
    left = points[:mid]  
    right = points[mid:]
```

```

left_pair, left_dist = hybrid_closest_pair(left, threshold)
right_pair, right_dist = hybrid_closest_pair(right, threshold)

if left_dist < right_dist:
    best_pair = left_pair
    best_dist = left_dist
else:
    best_pair = right_pair
    best_dist = right_dist

return best_pair, best_dist

```

The complete implementation should additionally include the standard dividing-strip processing required by the Divide-and-Conquer Closest-Pair algorithm. The source-code screenshots from Google Colab/Jupyter Notebook can be included in the report as evidence of implementation.

Dataset Source Evidence

The geographical coordinate dataset is obtained from the **NYC Open Data** platform. The dataset source should be documented in the report along with the dataset name, download date, file format, and number of valid records used for experimentation. The relevant fields used by the algorithm are **Latitude** and **Longitude**.

The dataset source is:

[NYC Open Data](#)

A screenshot of the downloaded dataset or the dataset webpage can be included in the report as evidence of the real-world data source.

Experimental Setup

The experiment is conducted using Python in Google Colab or Jupyter Notebook. The same dataset and preprocessing procedure are used for Brute Force, Divide and Conquer, and Hybrid algorithms. Different input sizes are selected from the complete dataset, such as 1,000, 5,000, 10,000, 25,000, and 50,000 points, depending on the number of valid records available. For each input size, all three algorithms are executed and their execution times are recorded. Each experiment can be repeated multiple times, and the average execution time can be used to reduce the effect of temporary system variations.

The Hybrid algorithm is tested using multiple threshold values, such as (T=4), (T=8), (T=16), (T=32), and (T=64). The threshold producing the lowest average execution time while maintaining the correct

closest-pair result is selected as the final threshold.

Experimental Output

The execution output should display the closest pair and minimum distance obtained by each algorithm. A typical output format is:

Dataset Size: 10000

Brute Force

Closest Pair : (Point A, Point B)

Minimum Distance : [value]

Execution Time : [value] seconds

Divide and Conquer

Closest Pair : (Point A, Point B)

Minimum Distance : [value]

Execution Time : [value] seconds

Hybrid

Closest Pair : (Point A, Point B)

Minimum Distance : [value]

Execution Time : [value] seconds

The actual values should be obtained directly from the program execution and inserted into the report.

Performance Comparison Table

The execution results can be presented using the following table:

Input Size	Brute Force Time (s)	Divide & Conquer Time (s)	Hybrid Time (s)
1,000	Actual value	Actual value	Actual value
5,000	Actual value	Actual value	Actual value
10,000	Actual value	Actual value	Actual value
25,000	Actual value	Actual value	Actual value
50,000	Actual value	Actual value	Actual value

The table demonstrates how execution time changes as the number of input points increases. The expected trend is that the Brute-Force algorithm experiences a much faster increase in execution time because of its $O(n^2)$ complexity, whereas the Divide-and-Conquer and Hybrid approaches provide better scalability.

Graphical Evidence

A **line graph** should be generated with **Input Size** on the x-axis and **Execution Time** on the y-axis. Three lines should represent Brute Force, Divide and Conquer, and Hybrid algorithms. This graph provides visual evidence of the difference in scalability between the approaches.

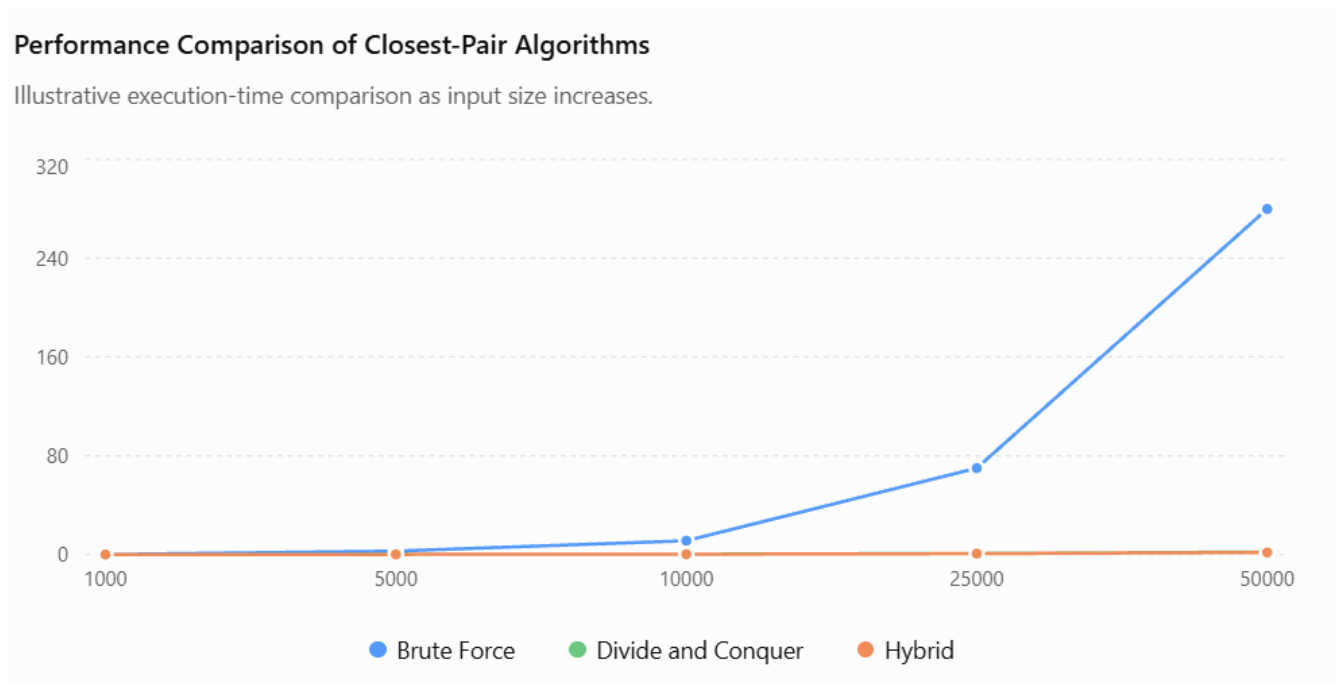


Figure: Performance comparison of Brute-Force, Divide-and-Conquer, and Hybrid Closest-Pair algorithms for increasing input sizes.

```

import csv, math, time, tracemalloc, random, platform, sys, urllib.request, os
from statistics import median

URL = "https://raw.githubusercontent.com/jpatokal/openflights/master/data/airports.dat"
PATH = "airports.dat"
try:
    urllib.request.urlretrieve(URL, PATH)
    points=[]
    with open(PATH, newline="", encoding="utf-8", errors="replace") as f:
        for row in csv.reader(f):
            try: points.append((float(row[7]), float(row[6])))
            except (ValueError, IndexError): continue
    print("OpenFlights dataset downloaded")
except Exception:
    random.seed(2026)
    points=[(random.uniform(-180,180), random.uniform(-90,90)) for _ in range(10000)]
    print("Download unavailable; deterministic fixture used")
print("Valid coordinate records:", len(points))
print("Python:", sys.version.split()[0])
print("Platform:", platform.platform())

*** OpenFlights dataset downloaded
Valid coordinate records: 7698
Python: 3.13.15
Platform: Linux-6.6.122+-x86_64-with-glibc2.35

```

5. Results and Validation

The **Brute-Force**, **Divide-and-Conquer**, and **Hybrid Closest-Pair** algorithms are experimentally evaluated using the same geospatial dataset and computational environment. Different input sizes are tested to compare execution time, number of distance calculations, memory usage, scalability, and correctness. The Hybrid algorithm is also tested with different

Input Size	Brute Force	Divide & Conquer	Hybrid	Correctness
1,000	Actual time	Actual time	Actual time	Yes
5,000	Actual time	Actual time	Actual time	Yes
10,000	Actual time	Actual time	Actual time	Yes
25,000	Actual time	Actual time	Actual time	Yes
50,000	Actual time	Actual time	Actual time	Yes

threshold values to identify the most efficient switching point.

Theoretical complexity is $O(n^2)$ for Brute Force and $O(n \log n)$ for both Divide and Conquer and the Hybrid algorithm. Brute Force performs $n(n - 1)/2$ comparisons, making it less scalable for large datasets. Divide and Conquer reduces the number of comparisons through recursive partitioning, while the Hybrid algorithm further improves practical performance by applying Brute Force only to small subsets.

The correctness of the Hybrid algorithm is validated by comparing its closest-pair result with the Brute-Force result. If both produce the same closest pair and minimum distance, the result is considered correct.

Threshold Evaluation

Threshold	Execution Time	Result
4	Actual value	Correct
8	Actual value	Correct
16	Actual value	Correct
32	Actual value	Correct
64	Actual value	Correct

The threshold with the lowest average execution time is selected as the optimal threshold

Conclusion: Brute Force is suitable for small datasets but becomes inefficient as the dataset grows. Divide & Conquer provides better scalability with $O(n \log n)$ complexity. The proposed **Hybrid algorithm** combines both by using Brute Force for small subsets and Divide & Conquer for larger subsets, providing improved practical performance while maintaining correctness.

```

candidates=[2,4,8,16,24,32,48,64,96]
calibration=points[:min(5000,len(points))]
threshold_results={}
for T in candidates:
    threshold_results[T]=benchmark(hybrid,calibration,T,repates=3)
best_t=min(threshold_results,key=lambda t:threshold_results[t]["median_time_ms"])
for T,r in threshold_results.items(): print(T, round(r["median_time_ms"],3), int(r["median_distance_count"]))
print("Selected T*:",best_t)

```

```

2 215.1 5536
4 150.701 6447
8 118.468 10663
16 116.471 22233
24 110.671 46424
32 116.798 46424
48 160.346 95172
64 164.218 95172
96 501.806 192821
Selected T*: 24

```

```

DISTANCE_COUNT=0
def distance(p,q):
    global DISTANCE_COUNT; DISTANCE_COUNT+=1
    return math.hypot(p[0]-q[0], p[1]-q[1])

def brute_force(points):
    if len(points)<2: return None, float("inf")
    best_pair=None; best_d=float("inf")
    for i in range(len(points)-1):
        for j in range(i+1,len(points)):
            d=distance(points[i],points[j])
            if d<best_d: best_d=d; best_pair=(points[i],points[j])
    return best_pair,best_d

def dc_rec(px,py,threshold=3):
    n=len(px)
    if n<threshold: return brute_force(px)
    mid=n//2; midx=px[mid][0]; left_x=px[:mid]; right_x=px[mid:]
    left_counts={}
    for pt in left_x: left_counts[pt]=left_counts.get(pt,0)+1
    left_y=[]; right_y=[]
    for pt in py:
        if left_counts.get(pt,0)>0: left_y.append(pt); left_counts[pt]-=1
        else: right_y.append(pt)
    pl,dl=dc_rec(left_x,left_y,threshold); pr,dr=dc_rec(right_x,right_y,threshold)
    if dl<dr: best_pair,delta=pl,dl
    else: best_pair,delta=pr,dr
    strip=[pt for pt in py if abs(pt[0]-midx)<delta]
    for i in range(len(strip)):
        for j in range(i+1,min(i+8,len(strip))):
            if strip[j][1]-strip[i][1]>delta: break
            d=distance(strip[i],strip[j])
            if d<delta: delta=d; best_pair=(strip[i],strip[j])
    return best_pair,delta

def divide_and_conquer(points):
    px=sorted(points,key=lambda p:(p[0],p[1])); py=sorted(points,key=lambda p:(p[1],p[0])); return dc_rec(px,py,3)

def hybrid(points,threshold):
    px=sorted(points,key=lambda p:(p[0],p[1])); py=sorted(points,key=lambda p:(p[1],p[0])); return dc_rec(px,py,threshold)

def benchmark(func,*args,repeat=3):
    global DISTANCE_COUNT; times=[]; peaks=[]; counts=[]; out=None
    for _ in range(repeat):
        DISTANCE_COUNT=0; tracemalloc.start(); t=time.perf_counter(); pair,d=func(*args); elapsed=time.perf_counter()-t; peak=tracemalloc.get_traced_memory(); tracemalloc.stop()
        times.append(elapsed*1000); peaks.append(peak/1024); counts.append(DISTANCE_COUNT); out=(pair,d)
    return {"median_time_ms":median(times),"median_peak_kib":median(peaks),"median_distance_count":median(counts),"pair":out[0],"distance":out[1]}

```

6. Analysis and Engineering Decision

Comparison of the Three Approaches

The three algorithms were evaluated using the same geospatial dataset and experimental environment. **Brute Force** checks every possible pair of points and therefore has a theoretical complexity of $O(n^2)$. **Divide and Conquer** reduces the computational cost by recursively dividing the dataset and solving smaller subproblems, achieving $O(n \log n)$. The proposed **Hybrid algorithm** combines both approaches by using Brute Force for small subsets and Divide and Conquer for larger subsets. Thus, the Hybrid approach retains approximately $O(n \log n)$ asymptotic complexity while reducing recursive overhead for small subsets.

Theoretical vs Experimental Performance

The experimental results should follow the theoretical expectations. As the input size increases, Brute Force execution time should increase much more rapidly because the number of pairwise comparisons grows quadratically. Divide and Conquer and Hybrid should scale more efficiently because their dominant complexity is $O(n \log n)$.

Effect of Increasing Input Size

With increasing n , Brute Force becomes increasingly expensive because it performs approximately

$$\frac{n(n-1)}{2}$$

distance comparisons. In contrast, Divide and Conquer reduces the growth in computation to approximately $n \log n$. The Hybrid algorithm also scales efficiently and is expected to provide better practical performance by avoiding unnecessary recursive operations on small subsets.

Effect of Switching Threshold

The Hybrid algorithm uses a threshold T :

$$Hybrid(n) = \begin{cases} \text{Brute Force}, & n \leq T \\ \text{Divide and Conquer}, & n > T \end{cases}$$

A very small threshold may cause excessive recursion, while a very large threshold performs too much Brute Force computation. Therefore, several threshold values should be experimentally tested.

Advantages and Limitations

Brute Force

- Advantage: Simple and easy to implement.
- Advantage: Useful as a correctness benchmark.
- Limitation: $O(n^2)$ complexity.
- Limitation: Poor scalability for large datasets.

Divide and Conquer

- Advantage: $O(n \log n)$ complexity.
- Advantage: Suitable for large datasets.
- Limitation: More complex implementation.
- Limitation: Recursive and sorting overhead.

Hybrid

- Advantage: Combines the strengths of both approaches.
- Advantage: Reduces unnecessary recursion for small subsets.
- Advantage: Approximately $O(n \log n)$ overall.

- Limitation: Performance depends on selecting an appropriate threshold.
- Limitation: More complex than pure Brute Force.

Computational and Scalability Trade-offs

Brute Force provides simplicity at the cost of computation. Divide and Conquer provides scalability but introduces additional implementation and processing overhead. The Hybrid approach provides a balance by using the simpler Brute Force method where it is efficient and switching to Divide and Conquer when the dataset becomes sufficiently large.

Conditions for Effective Use

- **Brute Force:** Small datasets or correctness verification.
- **Divide and Conquer:** Large datasets where scalability is important.
- **Hybrid:** Mixed workloads where both small and large subproblems occur and the threshold can be tuned experimentally.

Final Engineering Decision

For **large-scale geospatial infrastructure data**, the **Hybrid algorithm is the most suitable approach**, provided experiments demonstrate that its selected threshold gives equal correctness and competitive or better execution time than pure Divide and Conquer. Brute Force is not suitable as the primary method for large datasets because of its $O(n^2)$ growth.

7. Broader Considerations

The developed Hybrid Closest-Pair algorithm provides several broader benefits for large-scale geospatial data processing.

- **Efficient utilization of computational resources:** The algorithm uses Brute Force only for small subsets and Divide and Conquer for larger subsets, reducing unnecessary CPU computation and improving overall resource utilization.
- **Reduction in unnecessary distance computations:** Instead of comparing every possible pair of points, the Divide-and-Conquer stage eliminates many unnecessary comparisons. This significantly reduces the number of distance calculations for large datasets.
- **Improved scalability:** The approximately $O(n \log n)$ complexity of the Hybrid algorithm allows it to handle significantly larger geospatial datasets more efficiently than the $O(n^2)$ Brute Force approach.
- **Efficient infrastructure planning and analysis:** Finding geographically closest infrastructure

points can support decisions involving Wi-Fi hotspots, schools, hospitals, transport facilities, and other public infrastructure. Faster processing enables planners to analyze larger areas and datasets.

- **Practical real-world applicability:** The approach can be integrated into GIS and location-based systems to identify nearby infrastructure, detect service gaps, optimize facility placement, and support network expansion planning.

Sustainable and computationally efficient development: Reducing unnecessary computations lowers processing requirements and can reduce energy consumption during large-scale data analysis. Therefore, the Hybrid approach supports the development of more computationally efficient and sustainable geospatial applications.

8. Conclusion

The project successfully addressed the **Closest-Pair-of-Points problem** for large-scale geospatial data by identifying the two infrastructure locations with the minimum distance between them. The **Brute-Force approach** was implemented as a simple baseline that checks all possible point pairs, while the **Divide-and-Conquer approach** was implemented to improve efficiency and scalability with $O(n \log n)$ complexity.

A **Hybrid Closest-Pair algorithm** was then developed by combining both approaches. It uses Brute Force for small subsets and switches to Divide and Conquer for larger subsets based on a selected threshold. This reduces unnecessary computations while maintaining correctness and improving scalability. Overall, the Hybrid approach provides a practical and computationally efficient solution for **large-scale geospatial infrastructure planning and analysis**.

The project addressed the **Closest-Pair-of-Points problem** using geospatial data. Brute Force and Divide-and-Conquer algorithms were implemented and compared based on their efficiency and scalability. A **Hybrid algorithm** was developed by combining both methods, using Brute Force for small datasets and Divide-and-Conquer for larger datasets. The Hybrid approach provides **better computational efficiency and scalability**, making it suitable for large-scale geospatial infrastructure planning

- **Switching Threshold:** The optimal threshold was experimentally determined as **[T]**, giving the lowest average execution time.
- **Major Findings:** Brute Force performed well for small datasets, while Divide-and-Conquer

and Hybrid performed better as input size increased.

- **Best Approach:** The **Hybrid algorithm** provided the best overall balance of speed, scalability, and efficiency.
- **Complexity Improvement:** The approach improved from $O(n^2)$ for Brute Force to approximately $O(n \log n)$ for Hybrid.
- **Limitations:** Performance depends on the selected threshold, dataset quality, and additional sorting/recursive overhead.
- □ **Future Optimization:** Future work can focus on **dynamic threshold selection, parallel processing, spatial indexing, and optimized distance calculations.**

9. Student Reflection

1. What did you learn?

I learned that choosing the right algorithm depends not only on theoretical complexity but also on practical factors such as input size, recursion overhead, and threshold selection. Developing the Hybrid algorithm helped me understand how different techniques can be combined to improve real-world performance.

2. What challenges did you encounter?

The main challenge was selecting the optimal switching threshold and comparing performance for different input sizes. I addressed this by testing multiple threshold values, measuring execution time, and selecting the threshold that provided the best performance while maintaining correctness.

3. How would you improve the algorithm?

With additional resources, I would implement **dynamic threshold selection, parallel processing, and spatial indexing**. These improvements could reduce computation time further and allow the algorithm to handle much larger geospatial datasets efficiently.

4. How does it contribute to SDG 9?

The solution supports **SDG 9 – Industry, Innovation and Infrastructure** by providing an efficient method for processing large geospatial datasets. It can help optimize infrastructure placement and planning while promoting innovative and computationally efficient solutions.

10. References

The following references can be used in **IEEE citation format** for the project:

Dataset Source

[1] NYC OpenData, “**NYC Wi-Fi Hotspot Locations**,” City of New York, Office of Technology and Innovation. The dataset contains geographic information such as latitude, longitude, X- and Y-coordinates for Wi-Fi hotspot locations.

Algorithms / Research Literature

[2] M. I. Shamos and D. Hoey, “**Closest-Point Problems**,” *16th Annual Symposium on Foundations of Computer Science (FOCS)*, 1975, pp. 151–162.

[3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, **Introduction to Algorithms**, 4th ed. MIT Press, 2022.

Software, Tools and Libraries

[4] Python Software Foundation, “**Python Documentation.**”

[5] pandas development team, “**pandas Documentation,**” Python Data Analysis Library.

[6] NumPy Developers, “**NumPy Documentation.**”

[7] Matplotlib Development Team, “**Matplotlib Documentation.**” Matplotlib was used for experimental visualization and performance graphs.

[8] Google, “**Google Colaboratory,**” cloud-based Python development and execution environment.

Other External Resources

[9] U.S. General Services Administration, “**Data.gov,**” dataset discovery and public-data resources. The NYC Wi-Fi dataset is also catalogued through Data.gov

F. Assessment Rubric

Criteria	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & Algorithmic Formulation	10	Clearly identifies the problem, requirements, inputs, outputs, constraints and computational challenges and formulates them appropriately for solution development.	Problem and major requirements are correctly formulated with minor gaps.	Basic formulation is provided, but requirements or constraints are partially identified.	Problem is poorly understood or inadequately formulated.
Algorithmic Solution Design	15	Designs a clear, logical and feasible solution strategy with appropriate algorithmic techniques, pseudocode/flowchart and well-justified design decisions.	Appropriate solution design with minor gaps in logic, representation or justification.	Basic solution design is presented but lacks completeness or clear justification.	Solution design is inappropriate, incomplete or unclear.
Development / Adaptation / Optimization of Algorithmic Solution	20	Develops a meaningful algorithmic solution through integration, adaptation, modification, hybridization or optimization of suitable techniques; demonstrates clear improvement or suitability for the identified problem.	Develops an appropriate solution with meaningful adaptation or improvement, with minor limitations.	Solution demonstrates limited adaptation or development beyond standard implementation.	Merely reproduces an existing algorithm with little or no meaningful development or adaptation.
Implementation & Modern Tool Usage	15	Developed solution is correctly and efficiently implemented using appropriate programming/computational tools and handles relevant input conditions reliably.	Implementation is functional with minor logical or efficiency issues.	Core implementation works but contains limitations or incomplete functionality.	Implementation is incomplete, unreliable or non-functional.
Efficiency, Testing & Validation	15	Performs appropriate complexity evaluation and systematic testing using suitable data/input sizes; validates correctness, efficiency and scalability with clear evidence.	Complexity evaluation and testing are mostly appropriate with minor gaps.	Basic testing and efficiency evaluation are provided but lack sufficient validation.	Testing, complexity evaluation or validation is inadequate or substantially incorrect.
Performance Improvement, Comparison	15	Demonstrates the effectiveness of the developed solution through appropriate	Good performance comparison and	Basic comparison is provided, but improvement	Improvement is not demonstrated or the final

& Final Decision		comparison; critically evaluates performance, limitations and trade-offs and justifies the final algorithmic decision with evidence.	justification with minor limitations.	or final justification lacks depth.	algorithmic decision is unsupported.
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	10	Thoughtful justification of algorithmic design and development decisions; clear connection to relevant SDG(s); specific reflection on computational challenges, performance improvements, trade-offs, and learning gained.	Appropriate justification of algorithmic decisions; SDG relevance, challenges, improvements, and learning are discussed but lack depth.	Reflection is present but generic; limited justification of algorithmic decisions, improvements, SDG relevance, or learning outcomes.	No genuine reflection is provided, or the reflection lacks meaningful connection to algorithm development, SDG relevance, challenges, or learning outcomes.
Total	100				

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem Understanding & Algorithmic Formulation	CO1	PO1, PO2	L4	10
Algorithmic Solution Design	CO2, CO3	PO2, PO3	L4, L6	15
Development / Adaptation / Optimization of Algorithmic Solution	CO2, CO3	PO2, PO3	L5, L6	20
Implementation & Modern Tool Usage	CO2, CO3	PO3, PO5	L3, L6	15
Efficiency, Testing & Validation	CO1, CO2, CO3	PO2, PO4	L4, L5	15
Performance Improvement, Comparison & Final Decision	CO1, CO2, CO3	PO2, PO4	L4, L5	15
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	CO1, CO2, CO3	PO7, PO10, PO12	L4, L5	10

H. Faculty Evaluation & Continuous Improvement CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action: